

one trick pony - writeup

Fernando González

part 1: understanding

we're given a single python code which is the one that'll run when we netcat into it. it does the following:

1. asserts that `len(FLAG.strip("HTB{")) == 79`. this means `len(FLAG) == 84` (had to be careful cuz `flag.txt` cannot have newlines)
2. defines a pseudorandom function that generates bits based on the Legendre symbol of an ever-increasing random seed, modulo a 34 bit random prime p_{RNG} . pretty much like Damgård tells us in [1]. this PRF has a function to generate n of those bits
3. assigns a 259 bit **fixed** PRIME
4. defines a function to sign a message: it hashes it, passes it to an int mod PRIME (h), gets 500 bits from the PRF e , then calculates $h^e \bmod \text{PRIME}$
5. then it defines the menu logic: it first prints the random prime the PRF is using. then we have 2 functions: sign a message or verify an OTP. this OTP is `len(FLAG)*8 == 672` bits long taken from the PRF, and we have to send the bit sequence that matches it perfectly.
6. there's also a limit of up to 2000 sign operations we can do before the program auto terminating

part 2: observations

1. we can gather up to $1999 \cdot 500$ bits of encoded Legendre sequences
2. apparently this Legendre PRF is considered computationally secure
3. the Legendre sequence has the entire prime as period, no way to have less
4. the PRF skips 0, which is the only number with Legendre symbol 0. even then we wouldn't notice because it'd just output the same bit as the ones who have symbol -1
5. $p_{RNG} = 2^{34} \approx 1,7 \times 10^{10}$ is big, but not that big.
6. $\text{PRIME} - 1$ is smooth, many small factors (!!!)

part 3: getting there

as soon as i found out that $p - 1$ is smooth for a prime modulus associated with exponentiation, i went ahead and tried out factorizing with Pohlig-Hellman with sagemath. i married with a single message as input ("a") to have a fixed base g . then:

```
Fp = GF(p)
g = Fp(g)    # hash

# smooth!!!
factors = factor(p - 1)

y = Fp(y)    # my signature
# Pohlig-Hellman
x = discrete_log(y, g, ord=p-1)
```

and like magic, i could gather the Legendre sequence for that specific message. however, something tragic is there doesn't seem to be some sort of inverse function that takes that sequence and gives me where it starts. also there's no clear transition between $L(x)$ and $L(x+1)$.

after much doubting i resorted to making a direct bruteforce of that sequence. first i tried making a checker that, for each seed, checks if the next 500 bits of consecutive "Legendre bits" and are the same to the ones of the given x .

p.s: the flag mentioned something about using meet in the middle. Laaksonen [2] tells us in his competitive programming book:

Meet in the middle is a technique where the search space is divided into two parts of about equal size. A separate search is performed for both of the parts, and finally the results of the searches are combined.

i'm interested to know what could've been done to further reduce the time complexity with a technique like this

references

[1] https://link.springer.com/chapter/10.1007/0-387-34799-2_13

[2] <https://cses.fi/book/book.pdf>